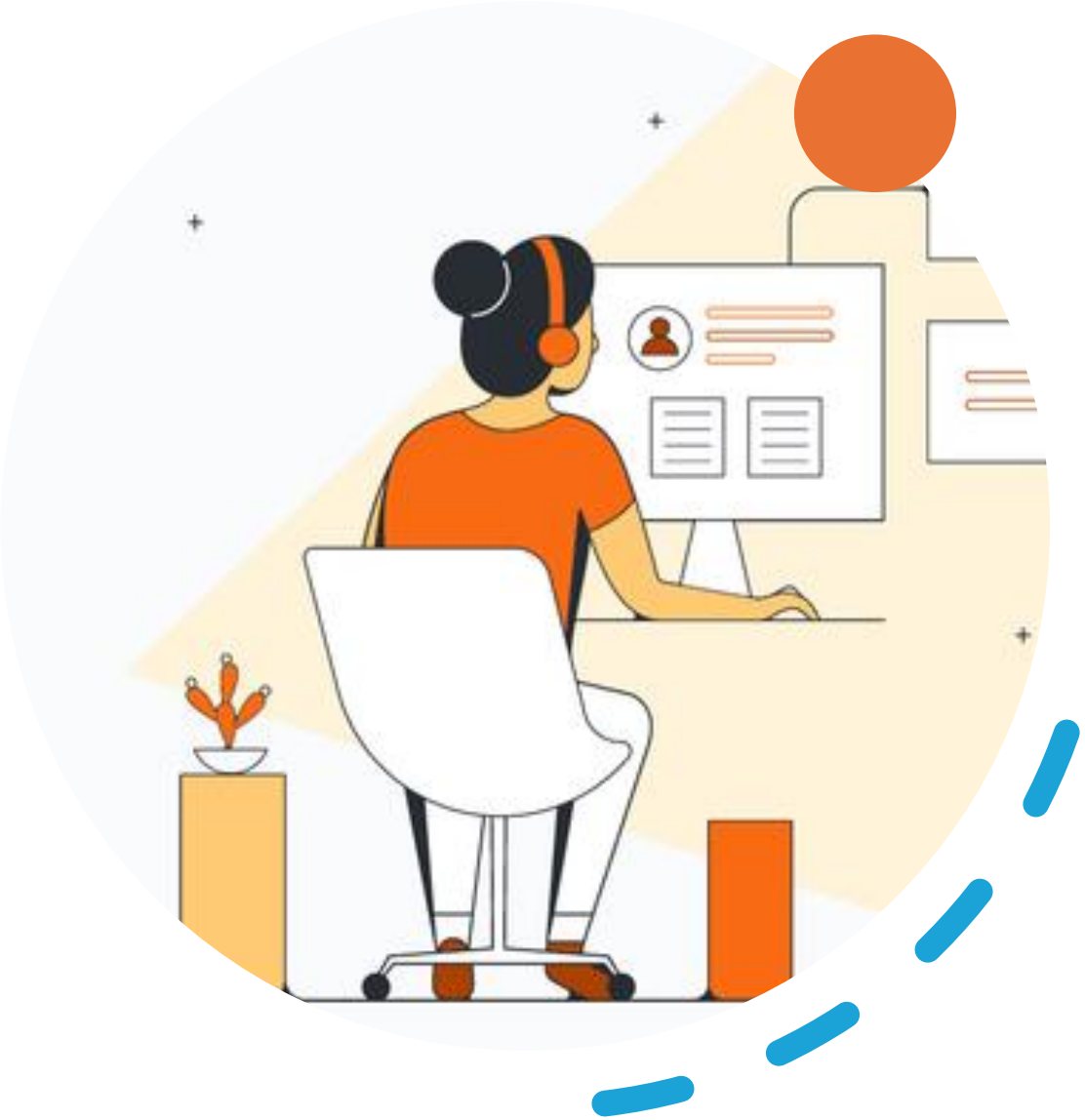
Patrones de diseño

Emmanuel Cetina Gomez

El Contexto del Diseño

Diseñar no es programar directamente; es el puente entre los requerimientos y el código. Implica tres tareas fundamentales:

- **Descomponer:** Dividir el sistema en piezas (módulos/clases).
- **Asignar:** Definir las tareas de cada pieza.
- **Conectar:** Establecer relaciones y dependencias.



¿Qué es un Patrón de Diseño?

Es una solución probada a un problema recurrente. No es código estático, sino un "molde" universal adaptable a cualquier lenguaje.

¿Para qué sirven?

Vocabulario Común: Permite al equipo entenderse rápido con una palabra (ej. "Usa un Singleton") en vez de explicar todo el código.

Gestión del Cambio: Hacen que el sistema sea más fácil de modificar en el futuro sin romperse.

No reinventar la rueda: Usas soluciones que expertos ya perfeccionaron

Patrón Singleton

Objetivo: Garantizar que una clase tenga una **única instancia** en toda la aplicación.

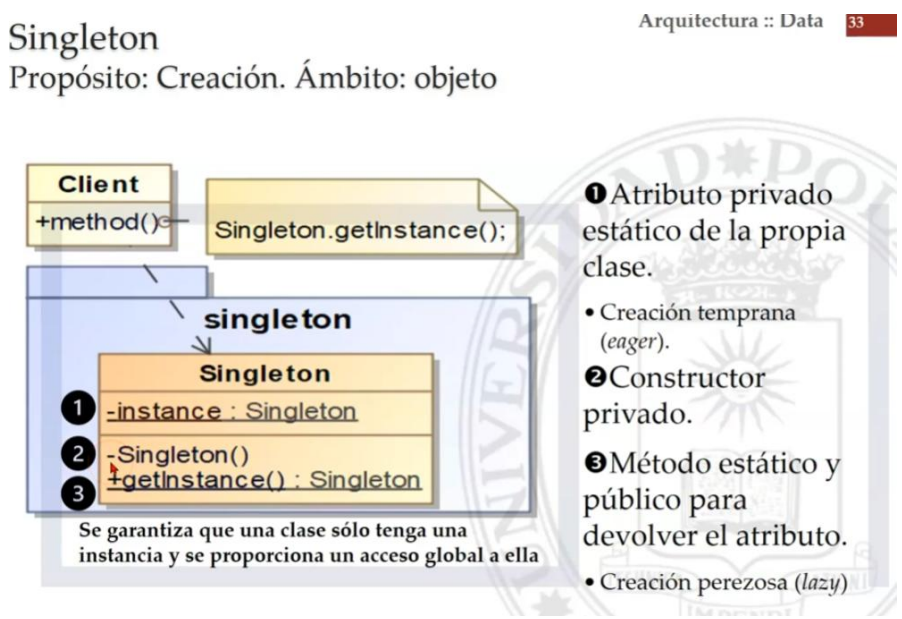
- **Uso común:** Conexiones a bases de datos, sistemas de log o colas de impresión.
- **Ventaja Clave:** Ahorro de memoria y control centralizado de recursos.

App.java | Conexion.java | Singleton/pom.xml

```java
1  package com.mitocode.model;
2
3  public class Conexion {
4
5      //Declaración
6      private static Conexion instancia;
7      //private static Conexion instancia = new Conexion();
8
9      //Para evitar instancia mediante operador "new"
10     private Conexion() {
11
12     }
13
14     //Para obtener la instancia unicamente por este metodo
15     //Notese la palabra reservada "static" hace posible el acces
16     public static Conexion getInstancia() {
17         if(instancia == null) {
18             instancia = new Conexion();
19         }
20         return instancia;
21     }
22
23     //Método de prueba
24     public void conectar() {
25         System.out.println("Me conecté a la BD");
26     }
27
28     //Método de prueba
29     public void desconectar() {
30         System.out.println("Me desconecté de la BD");
```

Cómo se hace un Singleton

- **Atributo Estático:** Una variable interna donde se guarda la única instancia.
- **Constructor Privado:** Para prohibir que nadie haga new Clase() desde fuera.
- **Método getInstance:** La única puerta pública para pedir la instancia.



Variantes del Singleton

Creación Temprana (Eager)

- **Mecánica:** La instancia se crea al inicio.
- **Riesgo:** Puede ralentizar el arranque al cargar recursos no utilizados.

Creación Perezosa (Lazy)

- **Mecánica:** Se crea solo al llamar a getInstance().
- **Ventaja:** Optimiza el arranque y ahorra memoria; si no se usa, nunca se crea.